REGULATORY & SUPPORT COPILOT · PART GUIDE 4 OF 5

Part 4: Request Desk Agent and Orchestrator

You build the Request Investigator that answers bank support tickets and student requests from records and documents, and the orchestrator that runs every task, step by step, for every template.

Owner	(write your name)
Your folder	`part4_request_desk/` (inside it: `investigator.py`, `prompts/`, `orchestrator/`, `tests/`) · shared with Part 5: `tests/integration/`
Your reviewer	Part 5 owner · you review Part 3
Read first	00-Project-Guide.pdf (Sections 3, 4, 8, 11, 12, 14)

Contents

1. Your role
2. What you deliver
3. Dependencies
4. Step-by-step plan
5. The Request Investigator
6. The orchestrator
7. Quality: how you will be measured
8. How your work shows on the website
9. Testing and definition of done
10. GitHub issues to create
11. Starter prompt for your AI tool
12. Common mistakes

1. Your role

Two jobs that make you the team's **integrator**:

1. **Request Investigator (Workflow B, "Request Desk"):** read an incoming request (a client bank asking why a payment was rejected, a student asking why registration failed), find the record it is about, identify the reason, gather evidence from the organization's documents, and draft a clear reply

for a human to approve. The same code works for every template; only the template's settings change.

2. **Orchestrator:** the "manager" that receives a task from the website, runs the right agents in order in the background, records each step's status and output, writes audit events, and stores everything in SQLite so the website can show progress live.

> **Guide's note:** You are where the parts meet, so you will see integration problems first. Don't fix other people's code yourself; open an Issue for them with the exact error and input. Your first-integration demo in week 4 is the most important milestone of the project. Never write the words "bank", "payment", "student" or "course" into your code: they come from the template.

2. What you deliver

#	Deliverable	Location
D1	Request Investigator	`part4_request_desk/investigator.py`, `part4_request_desk/prompts/investigator.md`
D2	Task store (SQLite)	`part4_request_desk/orchestrator/store.py`
D3	Orchestrator: start tasks, run steps in background, update status	`part4_request_desk/orchestrator/orchestrator.py` (re-exported in `orchestrator/__init__.py`)
D4	Command-line runner	`part4_request_desk/orchestrator/run.py`
D5	Integration tests for both workflows and both templates (with Part 5)	`tests/integration/`
D6	Tests, READMEs, AI_LOG	`part4_request_desk/tests/`, `part4_request_desk/README.md`, `part4_request_desk/AI_LOG.md`
D7	Your contract models: `Investigation`, `TaskStep`, `Task`	`contracts/schemas.py` (Contract Day)

3. Dependencies

You use	From	Until ready, use
`llm.complete_json()`, `search()`, `prompting.render()`	Part 2	`fake` provider, search stub, plain prompt text
`load_template()`, `search_records()`, `get_request()`, `list_requests()`	Part 1	stubs + first data sample
`summarize()`, `analyze()`, `plan()`	Part 3	their stubs

You use	From	Until ready, use
`audit.log_event()`, `mask()`	Part 5	stubs (do nothing / return text)

Uses your work	How
Part 5 (website)	`start_change_task()`, `start_request_task()`, `get_task()`, `list_tasks()`, `set_task_status()`, `update_output()`, `retry_task()`
Part 5 (evaluation)	`run_change_sync()` and `run_request_sync()`, the same steps without a background thread

4. Step-by-step plan

Week 1

1. Environment setup. Read the whole Project Guide carefully; you need the full picture more than anyone. Open both `template.yaml` files in `part1_templates/templates/` (or the spec in the Part 1 guide) and compare them.
2. Write `Investigation`, `TaskStep` and `Task` for Contract Day. Propose the `outputs` keys (Section 6.3) to the team.

Week 2

3. Merge stubs: `investigate()` returns the example; the orchestrator functions exist and return a fake finished task.
4. Implement `orchestrator/store.py` for real (it has no dependencies).

Week 3

5. Implement the real orchestrator with background threads, using the other parts' stubs. Test with `python -m part4_request_desk.orchestrator.run`.
6. Implement the Request Investigator with the `fake` provider on the payments template (the main one).

Week 4: first integration (milestone M2)

7. Connect the real parts of Parts 1–3 as they become ready. Sit with Part 5 and make both workflows run from the website with the `fake` provider.
8. Write the integration tests for the payments template, then copy them for the university template.

Weeks 5–6

9. Switch to a real model. Run all requests of both templates, compare with each `request_eval.json`, and improve the investigator.

10. Error handling: a failing step marks the task failed with a readable message; a **Retry** runs the task again.

Weeks 7–8

11. Report section, demo rehearsal (Demos B and C are yours), swap session.

5. The Request Investigator

```
investigate(request: Request) -> Investigation
```

Everything specific to an organization comes from the template: `template = load_template(request.template_id)`. You use `template.item_pattern`, `template.reason_codes_doc`, `template.prompt_hints`, `template.labels` and the template's connector (through Part 1's `search_records`).

Template	Item (what a record is about)	`item_pattern`	Example request
payments (main)	Payment reference	`(?:payment|ref|transfer)\s*[#=]?\s*(\d{4,6})` (case-insensitive; group 1 is the item)	P-REQ-007 from BANK-C: "Why was payment 7731 rejected?"
university	Course code	`[A-Z]{2,4}\d{3}`	U-REQ-007 from S-2291: "Why can't I register for SE301?"

Steps (code first, LLM where needed):

1. **Find the item.** First with code: `re.search(template.item_pattern, request.body, re.IGNORECASE)`; if the pattern has a capture group, the item is group 1 ("payment 7731" → `7731`), otherwise the whole match (`SE301`). Use `request.item` if it is already set. Only if code finds nothing, ask the LLM to extract it (or say there is none), and check the LLM's answer against the same pattern.
2. **Classify the request.** If there is no item and the text is a general question (for example "What is the cheque cut-off time?" or "What is the absence limit?"), take the **question path**: answer from `search(request.body, filters={"doc_type": "document"}, template_id=request.template_id)`, with `reason_code = null` and no records.
3. **Search the records (code).** `search_records(person_id=request.person_id, item=item, template_id=request.template_id)`, for example `search_records(person_id="BANK-C", item="7731", template_id="payments")`.
   - **No item but a person:** search all of that person's records, keep those whose `unit_id` matches a unit named in the text (match the unit names and IDs from `list_units()`, for example "instant payment" → `IPS`, "internship" → `CS-BSc`), and take the **latest failed record** (a record with a reason code). This covers tickets such as "our instant transfers this morning keep failing", which name a module but no reference.

- **No records found** (for example a payment reference that does not exist) → `escalate = true`, `confidence = "low"`, and a reply that politely says the team is checking the reference.
- **The latest record is a success** (no reason code: a settled payment, a completed registration) → explain that it succeeded, with the time ("Payment 8820 was settled at 10:42 on 2 March").
- **A failed record** (rejected payment, blocked registration) → take its `reason_code` (for example `AC04` closed account, or `PREREQ_MISSING`). Keep the matching records as `evidence`.

4. **Gather knowledge (RAG).** Search the template's reason-code document for the code: `search(reason_code, filters={"doc_type": "reason_codes", "reason_code": reason_code}, k=2, template_id=request.template_id)` (Part 2 indexes `reason_codes_doc` one chunk per code), which returns the section explaining the code and what to do next. Also `search(request.body, filters={"doc_type": "past_request"}, k=2, template_id=request.template_id)` for similar past cases. Put both in `sources`.
5. **Ask the LLM** for `root_cause` (one or two sentences) and `draft_reply`, giving it the request, the record evidence and the sources inside tags. Build the system prompt from `prompts/investigator.md` with `prompting.render()`, so `{organization}`, `{person_label}`, `{record_label}` and `{prompt_hints}` are filled from the template. Reply rules (write them in the prompt file): polite and short; explain the reason in plain words; tell the person what to do next; never include full account numbers, other clients' or people's data, or internal system names; sign exactly as the template's `prompt_hints` says ("Support Team, Cedar Payment Systems" or "Registrar's Office").
6. **Confidence (code, not LLM):** `high` = record found and its reason code explained in the documents · `medium` = record found but the code is not in the documents, or there are no sources · `low` = no record found. Set `escalate = true` for `low`.

6. The orchestrator

6.1 Public functions

```
start_change_task(doc: ChangeDocument) -> str        # returns task_id immediately
start_request_task(request: Request) -> str
get_task(task_id) -> Task
list_tasks(template_id=None, type=None, status=None) -> list[Task]
set_task_status(task_id, status) -> None             # called by the website after review
update_output(task_id, key, value) -> None           # e.g. store an edited action item
retry_task(task_id) -> str                           # starts a new task with the same input
run_change_sync(doc) -> Task                         # same steps, no thread (for evaluation)
run_request_sync(request) -> Task
```

6.2 How a task runs

1. `start_…` creates a `Task` with `status = "running"`, the input's `template_id`, all its steps `waiting`, saves it, writes an audit event `task_created`, starts a background thread (`threading.Thread(daemon=True)`), and returns the `task_id` (for example `T-20261103-0007`).
2. The thread runs the steps in order. For each step: set it `running` and save → call the agent → save the output under its key → set it `done` with timing → write an audit event `step_done` (actor = agent name, details = short summary such as "found 3 requirements").

3. If a step raises an exception: set it `failed` with the error message, set the task `failed`, write an audit event `step_failed`, and stop.
4. After the last step: set the task `pending_review`. From now on, only the website (human decisions) changes the status.

6.3 Steps and output keys

Task type	Steps (names shown on the website)	`outputs` keys
change	`Change Watcher` → `Impact Analyst` → `Action Planner`	`summary`, `impact_report`, `action_items`
request	`Request Investigator`	`investigation`

Store outputs as `model_dump(mode="json")` so they are plain JSON. To show finer progress for requests, `investigate()` can accept an optional `on_progress(message)` callback; the orchestrator saves the latest message in the step's `message` field ("Searching records …").

6.4 The task store (`part4_request_desk/orchestrator/store.py`)

- SQLite file from `settings.app_db`. One table `tasks(task_id TEXT PRIMARY KEY, template_id TEXT, type TEXT, status TEXT, created_at TEXT, data TEXT)`, where `data` holds the whole `Task` as JSON.
- Functions: `save_task(task)`, `load_task(task_id)`, `query_tasks(template_id=None, type=None, status=None)` (newest first).
- **Open a new connection in every function** (SQLite connections must not be shared between threads), and use `with conn:` so each write is committed.
- Part 5 keeps its audit and review tables in the same database file but in their own tables; agree on table names to avoid clashes.

6.5 Command-line runner

```
python -m part4_request_desk.orchestrator.run --change CHG-P-002
python -m part4_request_desk.orchestrator.run --request P-REQ-007
python -m part4_request_desk.orchestrator.run --request U-REQ-007 --template university
python -m part4_request_desk.orchestrator.run --list
```

It runs the task synchronously and prints each step and the final outputs. `--template` overrides `ACTIVE_TEMPLATE`. Everyone on the team uses it to test the whole chain without the website.

7. Quality: how you will be measured

Part 5's evaluation compares investigations with each template's `request_eval.json` (payments: 15 requests, university: 10):

Metric	Target
Item found correctly	≥ 95%
Reason code correct	≥ 90% (it comes from records, so code should make it near perfect)
Escalation correct (requests with no record escalate, others don't)	100%
Root cause contains the expected keywords	≥ 80%
Reply contains no full account number, IBAN, national ID or phone number	100%
Demo requests: P-REQ-007 → `AC04` (closed account); U-REQ-007 → `PREREQ_MISSING` (SE201 missing)	Always correct
Average time per request	Report it (no target)

8. How your work shows on the website

- The **Task page** step timeline (waiting / running / done / failed, with durations) comes straight from your `TaskStep`s; the website polls `get_task()` every 2 seconds.
- The **investigation card** shows the item, reason code, root cause, evidence records, sources, a confidence badge, an escalation warning, and the draft reply with Approve / Edit / Reject, all with the active template's labels.
- The **Dashboard** lists tasks from `list_tasks(template_id=<active>)`, so switching the template shows that organization's tasks.
- The **Retry** button calls `retry_task()`.

9. Testing and definition of done

- [ ] Investigator unit tests (fake provider, mocked records and search) for **both templates**: failed record (rejected payment, blocked registration), successful record (settled payment), no record (unknown reference), general question, and the person + unit fallback.
- [ ] Item-extraction tests for at least 6 writing styles per template, using each template's `item_pattern`.
- [ ] Confidence and escalation rules tested in code.
- [ ] Store tests: save/load round trip, filters (including `template_id`), newest first.
- [ ] Orchestrator tests: steps go waiting → running → done; a failing agent marks the task failed with its message; background start returns immediately.
- [ ] Integration tests (with Part 5): both workflows for both templates end in `pending_review` with all outputs, using the fake provider.
- [ ] `python -m part4_request_desk.orchestrator.run` works for every change and request of both templates with a real model.

10. GitHub issues to create

Label `part-4`:

1. Contract models: Investigation, TaskStep, Task (+ outputs keys proposal)
2. Stubs for investigator and orchestrator
3. SQLite task store with template_id
4. Orchestrator with background threads and step tracking
5. Command-line runner with --template
6. Investigator: item extraction from the template's item_pattern (regex + LLM fallback)
7. Investigator: record search, person + unit fallback, question path
8. Investigator: RAG sources, LLM reply with template hints, confidence rules
9. Error handling and retry
10. Integration tests for both workflows and both templates (with Part 5)
11. Quality run on both request_eval.json files

11. Starter prompt for your AI tool

The five ready prompts are in `part4_request_desk/PROMPTS.md`. The general shape is:

```
You are helping me build Part 4 (Request Desk Agent and Orchestrator) of a 5-person student
project, the "Regulatory & Support Copilot", a general product customized by templates
(a payments software company like ProgressSoft, and a university). I attached the project guide,
my part guide and contracts/schemas.py.
Rules: Python 3.11, Pydantic v2, sqlite3 and threading from the standard library; call the LLM
only through part2_knowledge.common.llm; use other parts only through their public functions
listed in Section 8.2 of the project guide; never hard-code template words (bank, payment,
student, course); edit only part4_request_desk/ and tests/integration/; pytest tests for every
function.
First, give me a short plan. Wait for my approval. Then do this task:
<for example: implement orchestrator/store.py as described in Section 6.4>.
```

12. Common mistakes

- **Running agents inside the web request.** The page freezes for a minute. Always use the background thread.
- **Sharing one SQLite connection between threads.** It crashes randomly. One connection per call.
- **Swallowing errors.** A failed step must show its message on the website, or nobody can debug it.
- **Letting the LLM find the reason code.** The records already contain it; code reads it exactly.
- **Hard-coding the payments template.** If your code only works for `BANK-C` and `7731`, the university demo will fail. Test both templates every week.
- **Waiting for everyone before integrating.** Integrate with stubs from week 3, then swap in real parts one by one.